

Capítulo 9

Circuitos Aritméticos

Circuitos aritméticos são sistemas digitais construídos utilizando lógica Booleana capazes de efetuar as quatro operações fundamentais da álgebra Euclideana. Tais sistemas são a base dos computadores e residem em um subsistema presente em todos os processadores conhecido como ULA - Unidade Lógica e Aritmética.

Neste capítulo serão utilizados os conceitos cobertos nos capítulos anteriores para o desenvolvimento de funções Booleanas e os circuitos digitais equivalentes capazes de efetuar cálculos aritméticos.

Primeiramente as quatro operações aritméticas em binário serão revisadas. A seguir será estruturada uma forma ingênua de implementação da soma binária em $\mathbb{N}$ que subsequentemente será estendida para atuar em $\mathbb{Z}$. A seguir uma análise de complexidade no tempo do somador será apresentada e em consequência uma versão mais eficiente será discutida. A seguir será apresentado o circuito capaz de efetuar a operação de subtração seguido de uma discussão de como a utilização do código de complemento de 2 para representação de quantidades negativas facilita a implementação da aritmética em álgebra Booleana.

A multiplicação binária é então discutida e apresentada sob a forma de um circuito que implementa a sucessão de somas. Uma forma eficiente para multiplicação por dois é discutida e o conceito é subsequentemente estendido para divisão por 2. Discute-se a seguir a implementação da divisão em $\mathbb{Z}$.

9.1 Somador Binário

Para que seja possível modelar circuitos digitais capazes de efetuar aritmética, faz-se necessário que se revise como as quatro operações aritméticas se operam no sistema numérico binário. Interessantemente, como tanto o sistema binário quanto o decimal são posicionais, as mesmas regras usadas nas operações aritméticas se aplicam.

A operação de soma em $\mathbb{N}$ visa agregar as quantidades de dois números em um único número. Intuitivamente, pode-se imaginar que os dois números a serem adicionados encontram-se colocados na reta natural como apresentado na figura 9.1. Note que a posição de "c" na linha se dá pela distância δ_2 de 0 a "b" aumentada pela distância δ_1 de 0 a "a".

Outra forma de pensar acerca da operação de soma refere-se a decrementar a posição de "a" em uma unidade ao mesmo tempo que se incrementa a posição de "b" em uma unidade repetindo o processo até que "a" seja deslocado para a posição 0 na reta. Esta na realidade é a abordagem utilizada na regra de soma comumente aprendida no ensino fundamental. Considere por exemplo a soma dos números $42_{10} \equiv 101010_2$ e $11_{10} \equiv 1011_2$.

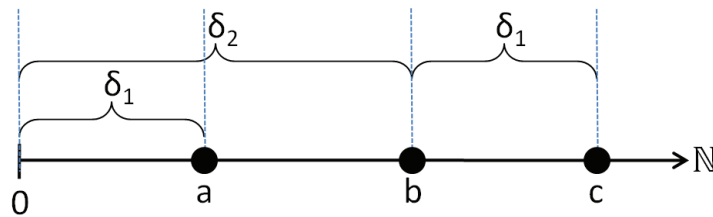


Figura 9.1: Exemplo da soma $c = a + b$ por localização na linha $\mathbb{N}$. A posição de "c" na linha se dá pela distância δ_2 de 0 a "b" aumentada pela distância δ_1 de 0 a "a".

			1		1	
	(5)	(4)	(3)	(2)	(1)	(0)
	1	0	1	0	1	0
+	0	0	1	0	1	1
	1	1	0	1	0	1

A regra para execução da operação de soma dita que se inicie o processo de adição da direita para a esquerda, ou seja, da casa menos significativa para a mais significativa. No caso da aritmética binária o processo de soma casa a casa é muito mais simples se comparado ao que ocorre no sistema decimal. Possivelmente apenas um decremento no sentido do 0 na reta em

$\mathbb{N}$ pode ocorrer, visto que o maior número possível no sistema binário é "1". Caso o processo de soma de uma casa binária resulte em um número maior que "1" (o único número maior que "1" passível de ser gerado pela soma de uma única casa binária será $2_{10} \equiv 10_2$) a casa imediatamente a esquerda deve ser incrementada em uma unidade e a casa corrente deve ser zerada. É possível capturar o comportamento de todas as possibilidades geradas pela soma binária de uma casa como demonstrado na tabela 9.1.

Tabela 9.1: Tabela verdade para a soma do bit menos significativo em $\mathbb{N}$.

a_0	b_0	s_0	co_0
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Para os demais bits deve-se considerar adicionalmente a possibilidade de que a casa anterior tenha gerado um "vai um", ou co_{i-1} onde i é o índice da casa em questão. Neste caso, para listar sistematicamente todas as possíveis combinações devem-se considerar três entradas, o bit da casa em questão do primeiro operando assim como do segundo operando e o bit de "vai um" possivelmente resultante da soma do bit anterior co_{i-1} renomeado neste caso ci_i . A tabela 9.2 apresenta todos os possíveis casos da soma dos bits subsequentes ao menos significativo. Note que no último caso, ou seja quando ambos os operandos e co_{i-1} são "1" a soma total acarreta em um 3_{10} o que significa que o bit de resultado s_i receberá zero e ainda ocorrerá um transbordo para o próximo bit.

Tabela 9.2: Tabela verdade para a soma do bit localizado na casa n em $\mathbb{N}$.

a_n	b_n	ci_{n-1}	s_n	co_n
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

A partir da tabela 9.1 é possível levantar a expressão booleana que implementa o circuito. Note que a saída S_0 se comporta como a porta OU-

EXCLUSIVO e que a saída co_0 é a operação E entre as entradas a e b . A equação define as saídas do circuito e a figura 9.2 o circuito equivalente.

$$\begin{aligned} s_0 &= a_0 \oplus b_0 \\ co_0 &= a_0 \cdot b_0 \end{aligned} \quad (9.1)$$



Figura 9.2: Diagrama de portas lógicas para o circuito que executa a soma do bit menos significativo, também conhecido como MEIO SOMADOR.

Com base na tabela 9.2 podemos levantar as expressões Booleanas que implementam o circuito somador dos demais bits. Note que ao mapear os bits da saída s_n para o mapa de Karnaugh obtêm-se a típica saída de "tabuleiro" que como se sabe não admite simplificação. Via manipulação algébrica obtêm-se:

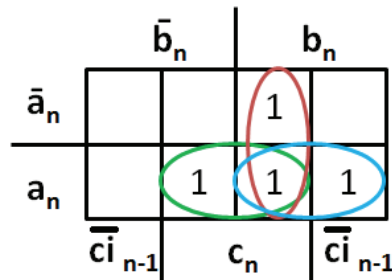
$s_i = \overline{a_i} \cdot \overline{b_i} ci_{i-1} + \overline{a_i} b_i \overline{ci_{i-1}} + a_i \overline{b_i} \cdot \overline{ci_{i-1}} + a_i b_i ci_{i-1}$ $\overline{a_i}(\overline{b_i} ci_{i-1} + b_i \overline{ci_{i-1}}) + a_i(\overline{b_i} \cdot \overline{ci_{i-1}} + b_i ci_{i-1})$ $\overline{a_i}(b_i \oplus ci_{i-1}) + a_i(\overline{b_i} \oplus \overline{ci_{i-1}})$ $\overline{a_i}X + a_i\overline{X}$ $a_i \oplus X$ $a_i \oplus b_i \oplus ci_{i-1}$	evidência $\overline{X}Y + X\overline{Y} = X \oplus Y$ $X = b_i \oplus ci_{i-1}$ $\overline{X}Y + X\overline{Y} = X \oplus Y$ $X = b_i \oplus ci_{i-1}$
---	--

Para a saída co_n o mapeamento para o mapa de Karnaugh permite simplificação como se pode observar no mapa apresentado na figura 9.3.

Aplicando o processo de simplificação obtêm-se as funções Booleanas apresentadas na equação 9.2.

$$\begin{aligned} s_i &= a_i \oplus b_i \oplus ci_{i-1} \\ co_i &= a_i ci_{i-1} + b_i ci_{i-1} + a_i b_i \end{aligned} \quad (9.2)$$

A realização direta das funções da equação 9.2 resultam no diagrama de portas lógicas apresentado na figura 9.4.

Figura 9.3: Mapa de Karnaugh para a saída co_n .

Os circuitos apresentados nas figuras 9.2 e 9.4 podem ser utilizados como subsistemas para compor somadores de tamanhos arbitrariamente grandes. Note que para o bit menos significativo o circuito do meio somador é suficiente pois não há a possibilidade de que haja um bit de carry. No entanto como será apresentado adiante é útil utilizar também para o primeiro bit um somador completo, pois tal adoção facilita a construção de somadores em $\mathbb{Z}$.

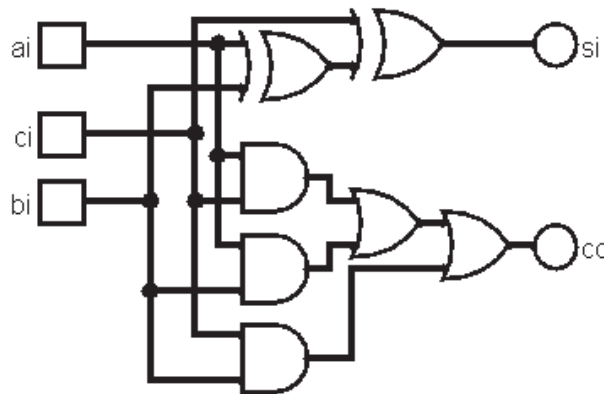


Figura 9.4: Diagrama de portas lógicas para o circuito que executa a soma dos demais bits, também conhecido como SOMADOR COMPLETO.

O circuito apresentado na figura 9.5 implementa utilizando os blocos do somador completo e do meio somador, um sistema digital capaz de somar dois números de quatro bits, $b_3b_2b_1b_0$ e $a_3a_2a_1a_0$, respectivamente.

Tal circuito é conhecido como somador de *ripple carry* pois a soma do bit imediatamente a esquerda depende do "vai um" ou *carry* vindo do bit imediatamente a direita. Consequentemente o bit mais significativo da soma só pode ser efetivamente computado após todos os bits menos significativos o terem sido.

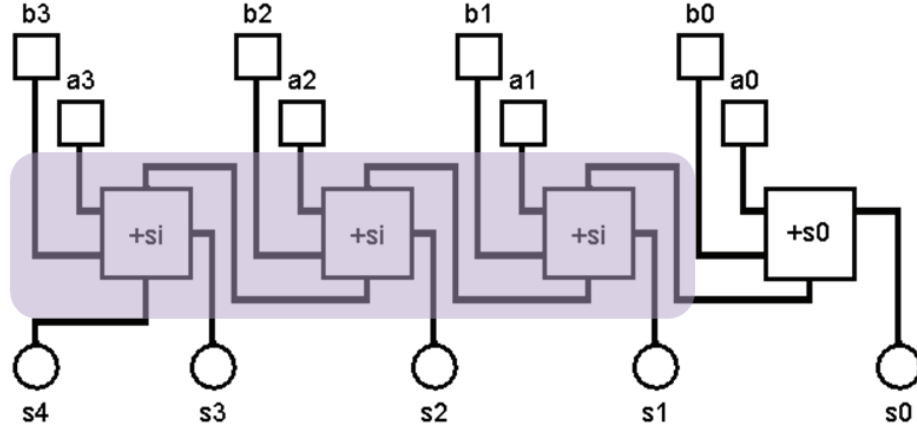


Figura 9.5: Somador de 4 bits construído utilizando o meio somador ($+s_0$) e o somador completos ($+s_i$).

Considerando que o somador *ripple carry* de n bits seja composto apenas por somadores completos e que o atraso de propagação das portas lógicas δ seja t é possível calcular quanto tempo demorará para que o somador completo compute um bit. Observa-se na figura 9.4 que o diagrama de portas lógicas é organizado em três níveis, conseqüentemente seu atraso será de $\delta = 3t$. O tempo necessário para que o somador compute a soma em sua completude será conseqüentemente $\delta_{total} = n \times 3t$. Observe que o tempo total de atraso do circuito depende diretamente do número de bits considerado na soma. Conseqüentemente quanto maior for o tamanho da palavra, maior será o atraso do circuito.

Há no entanto diversas outras formas em que o circuito somador pode ser realizado. Em especial uma solução comumente empregada para controlar o tempo de atraso é utilizar o que ficou convencionado chamar de *carry lookahead*.

Carry Lookahead - Esta implementação se fundamenta no fato de que via manipulação algébrica, é possível estimar muito cedo (em apenas $3t$) a possibilidade de que a soma de qualquer bit gere ou não um carry. Considere a equação 9.2 mais especificamente a expressão que computada o bit de "vai um" co_i . Esta expressão pode ser reescrita como na equação 9.3.

$$co_i = \underbrace{a_i b_i}_{g_i} + ci_{i-1} \underbrace{(a_i + b_i)}_{p_i} \quad (9.3)$$

O termo g_i refere-se a possibilidade do bit em consideração gerar por si só um "vai um" e o termo p_i considera a possibilidade de que um "vem um" no bit em consideração seja propagado como um bit de "via um". Note que tanto o termo g_i quanto p_i dependem única e exclusivamente dos bits sendo considerados, ou seja, podem ser gerados em $3t$ a partir do momento que as entradas A e B estejam disponíveis.

Uma expansão das equações individuais para cada um dos bits (a seguir) demonstra que é possível estimar qualquer bit de carry com base unicamente nos termos geradores e propagadores dos bits anteriores e do sinal de carry do primeiro bit. Note que na expansão fica claro que o tempo de propagação aumenta em apenas $1t$ para cada bit considerado em comparação com $3t$ na implementação ingênua (*ripple carry*).

$$c_1 = g_0 + p_0c_0 \quad (3t)$$

$$c_2 = g_1 + p_1g_0 + p_1p_0c_0 \quad (4t)$$

$$c_3 = g_2 + p_2g_1 + p_2p_1g_0 + p_2p_1p_0c_0 \quad (5t)$$

$$c_4 = g_3 + p_3g_2 + p_3p_2g_1 + p_3p_2p_1g_0 + p_3p_2p_1p_0c_0 \quad (6t)$$

$\vdots$

$$c_n = g_n + p_ng_{n-1} + p_np_{n-1}g_{n-2} \cdots + p_np_{n-1} \cdots p_0c_0 \quad (n+2 \text{ t})$$

Exemplo 9.1. Considere que se deseja construir um circuito somador de 32 bits. Deseja-se estimar qual será o atraso de propagação do circuito resultante. Para tal levam-se em consideração duas implementações possíveis, a estratégia *ripple carry* e *carry lookahead*. Estime qual será o atraso de cada uma das estratégias.

$$\textbf{ripple carry} \quad \rightarrow 1t + (n - 1) \times 3t = 1t + 31 \times 3t = 94t$$

$$\textbf{carry lookahead} \quad \rightarrow (n + 2)t = (32 + 2)t = 34t$$

Com base no exemplo 9.1 observa-se que a estratégia construtiva do *carry lookahead* é consideravelmente mais eficiente que *ripple carry*. Tal fato nos leva a considerar se a segunda opção é útil em alguma situação. De fato há um custo associado a implementação *carry lookahead* que é geralmente ignorado.

Considere a figura 9.6 que representa o circuito somador de palavras de 2 bits utilizando a estratégia de *carry lookahead*. Uma comparação direta com o somador completo (9.4) não torna evidente, mas fato é que o *carry lookahead* requer que mais conexões sejam feitas o que resulta em mais contatos (fios) estejam presentes no circuito. Considerando que circuitos integrados são construídos atualmente em superfícies esta estratégia construtiva leva a problemas sérios de roteamento das conexões visto que fios não podem se sobrepor em um plano o que em última instância leva a geração de circuitos que utilizam maior espaço físico. Circuitos espaçados implicam em maior

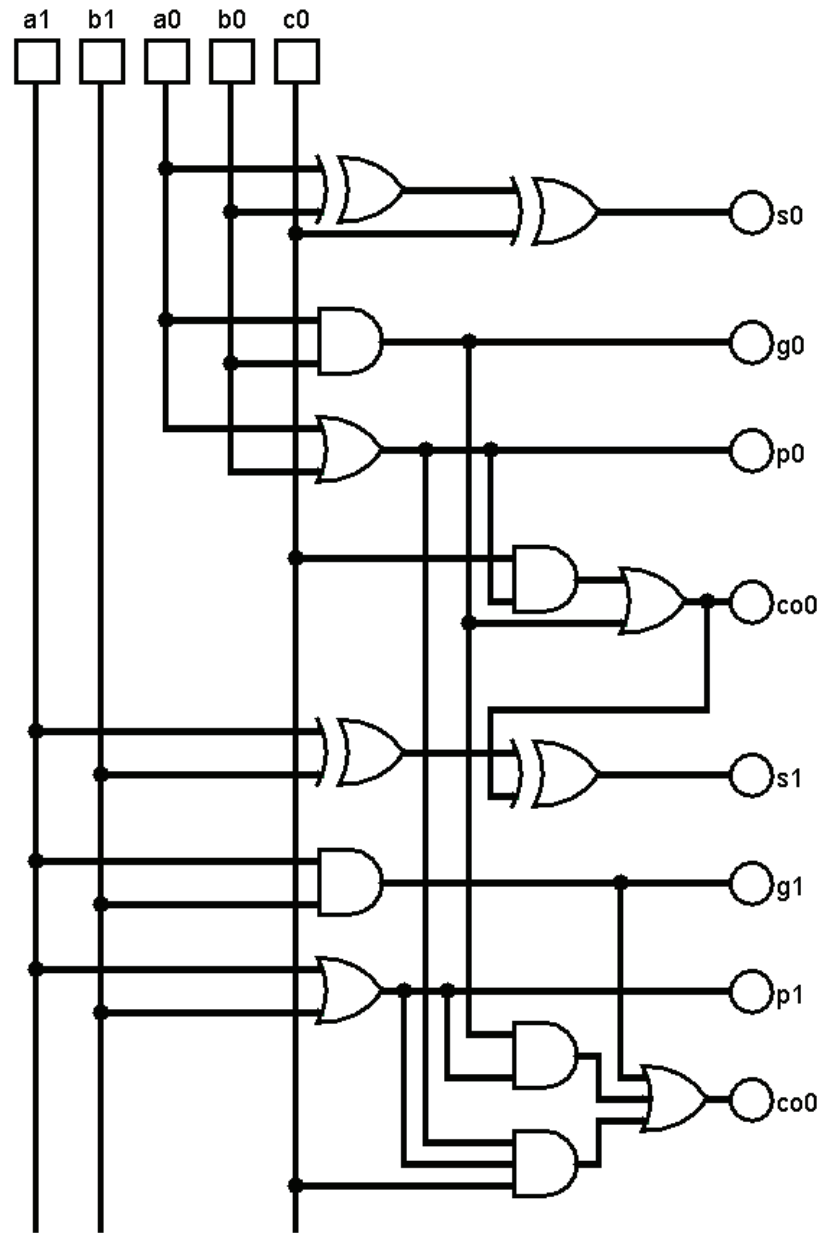


Figura 9.6: Somador de 2 bits construído utilizando a estratégia *carry lookahead*.

tempo de propagação do sinal o que por sua vez adiciona uma constante de proporcionalidade resultando na equação de tempo $n + 2 + \gamma t$ onde γ representa o atraso adicional de propagação entre conexões.

Ao considerar o conjunto dos números inteiros $\mathbb{Z}$ as coisas se tornam in-

$$\begin{array}{ll} F = X + Y & F = X + (-Y) \\ F = (-X) + Y & F = (-X) + (-Y) \end{array}$$

Caso 2 - $F = X + (-Y) \rightarrow$ Neste caso o problema admite que o resultado da soma seja negativo caso $X < Y$. Consequentemente é imprescindível que um sistema de codificação de números negativos seja adotado.

$$\begin{array}{r}
 \begin{array}{cccccccc}
 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & (+10) \\
 + & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & (-20) \\
 \hline
 1 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & (-10)
 \end{array}
 \end{array}$$

Ao considerar apenas a subtração de um bit não há a possibilidade de que tenha sido "emprestado" um "1" para a direita. Consequentemente apenas quatro casos devem ser considerados, nomeadamente $0 - 0 = 0$, $0 - 1 = 1$, $1 - 0 = 1$ e $1 - 1 = 0$. Como no segundo caso o número a ser subtraído é maior faz-se necessário que seja "emprestado" um "1" da casa mais a esquerda. Considere o bit menos significativo do exemplo a seguir no qual efetua-se a subtração de $42 - 11$.

$$\begin{array}{rcccccc}
& 1 & 1 & 1 & 1 & 10 \\
(5) & (4) & (3) & (2) & (1) & (0) \\
& 1 & 0 & 1 & 0 & 1 & 0 & (42) \\
- & 0 & 0 & 1 & 0 & 1 & 1 & (11) \\
\hline
& 0 & 1 & 1 & 1 & 1 & 1 & (31)
\end{array}$$

Nota-se que o bit menos significativo da subtração refere-se ao caso $0 - 1$. Consequentemente faz-se necessário que um "1" seja emprestado da casa imediatamente a esquerda o que valerá na primeira casa duas unidades ou 10_2 . Eis a razão porque $0 - 1 = 1$.

A tabela 9.3 sumariza o funcionamento da subtração do bit menos significativo.

Tabela 9.3: Tabela verdade para a subtração do bit menos significativo em $\mathbb{N}$.

a_0	b_0	s_0	t_0
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Com base na tabela é possível extrair as funções Booleanas que a implementam como apresentado na equação 9.4. Note que a função para o resultado tanto da soma como da subtração são o mesmo, ou seja, simplesmente um OU-EXCLUSIVO entre as entradas.

$$\begin{aligned} s_0 &= a_0 \oplus b_0 \\ t_0 &= \overline{a_0} \cdot b_0 \end{aligned} \tag{9.4}$$

A implementação via diagrama de portas lógicas das funções Booleanas da equação 9.4 são apresentadas na figura 9.7.

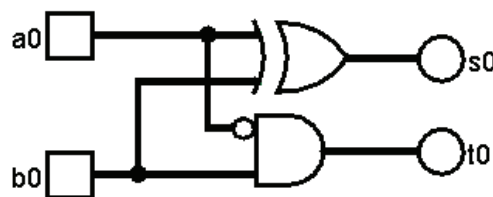


Figura 9.7: Diagrama de portas lógicas para o circuito que executa a subtração do bit menos significativo, também conhecido como MEIO SUBTRACTOR.

Para os demais bits deve admitir-se a possibilidade de que a casa em consideração tenha emprestado ou não um "1" para a casa a direita resultando na tabela verdade 9.4. Esta tabela não é tão óbvia quanto a tabela 9.2 referente a soma dos demais bits. O ponto a ser levado em consideração é

considerar cada casa individualmente assumindo a possibilidade de que um "1" tenha sido emprestado.

Tabela 9.4: Tabela verdade para a subtração do bit localizado na casa n em $\mathbb{N}$.

a_n	b_n	t_{n-1}	s_n	t_n
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Com base na tabela é possível extrair as funções Booleanas que a implementam como apresentado na equação 9.5. Apresenta-se como exercício a simplificação das equações canônicas $\sum_{s_n} = 1, 2, 3, 7$ e $\sum_{t_n} = 1, 2, 3, 7$.

$$\begin{aligned} s_i &= a_i \oplus b_i \oplus ci_{i-1} \\ co_i &= a_i ci_{i-1} + b_i ci_{i-1} + a_i b_i \end{aligned} \quad (9.5)$$

A realização direta das funções da equação 9.5 resultam no diagrama de portas lógicas apresentado na figura 9.8.

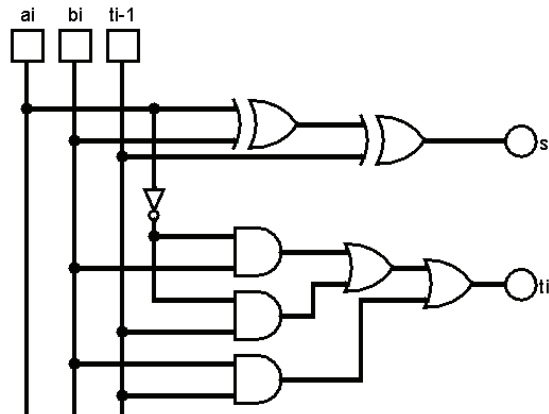


Figura 9.8: Diagrama de portas lógicas para o circuito que executa a subtração dos demais bits, também conhecido como SUBTRATOR COMPLETO.

9.3 Circuito Multiplicador

A multiplicação binária em $\mathbb{N}$ tal como a multiplicação decimal nada mais é que uma soma sucessiva. Mais especificamente soma-se o multiplicando tantas vezes quanto a quantidade do multiplicador. Numericamente, $10 \times 3 = 10 + 10 + 10 = 30$.

Um fato importante que deve ser notado refere-se ao número de casas produzido pela operação de multiplicação. Considere o caso decimal para números compostos por duas casas decimais. O caso extremo refere-se a $99 \times 99 = 9801$. Para três casas têm-se $999 \times 999 = 998001$. Nota-se claramente que o número de casas do resultado será igual a soma do número de casas do multiplicador e do multiplicando. O mesmo ocorre com números na base binária. Consequentemente, para palavras de 8 bits o resultado será composto no máximo or 16 bits, para palavras de 16 bits o resultado será no máximo 32 bits e assim sucessivamente.

Exemplo 9.2. Considere a multiplicação binária abaixo:

(101)										0	0	0	0	1	0	1	0
(102)									$\times$	0	0	0	0	0	0	1	1
(103)										0	0	0	0	1	0	1	0
(104)								$+$	0	0	0	0	1	0	1	0	$\downarrow$
(105)										0	0	0	1	1	1	1	0
(106)							$+$	0	0	0	0	0	0	0	0	$\downarrow$	$\downarrow$
(107)										0	0	0	0	0	1	1	0
(108)						$+$	0	0	0	0	0	0	0	0	0	$\downarrow$	$\downarrow$
(109)										0	0	0	0	0	1	1	0
(110)					$+$	0	0	0	0	0	0	0	0	0	$\downarrow$	$\downarrow$	$\downarrow$
(111)										0	0	0	0	0	1	1	0
(112)					$+$	0	0	0	0	0	0	0	0	0	$\downarrow$	$\downarrow$	$\downarrow$
(113)										0	0	0	0	0	1	1	0
(114)					$+$	0	0	0	0	0	0	0	0	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$
(115)										0	0	0	0	0	1	1	0
(116)					$+$	0	0	0	0	0	0	0	0	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$
(117)	"c"	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0

Como se pode notar ela implementa a multiplicação binária seguindo o mesmo conjunto de regras empregado na multiplicação inteira. Chama-se multiplicando o número de cima ($0000\ 1010_2$) e multiplicador o número de baixo ($0000\ 0011_2$). A tabela verdade 9.5 descreve o funcionamento da multiplicação bit a bit.

A multiplicação transcorre da seguinte maneira. Os bits do multiplicador são avaliados um a um da direita para a esquerda. Caso o bit seja "0" a multiplicação pelo multiplicando resultará em "0" e caso o bit seja "1" o resultado será exatamente o multiplicando. A seguir segue a seleção do próximo bit a esquerda e o processo se repete. Cada novo bit do multiplicador selecionado, o resultado deve ser deslocado em uma casa para a esquerda e os resultados intermediários adicionados.

Tabela 9.5: Tabela verdade para a multiplicação bit a bit em $\mathbb{N}$.

a_i	b_i	s_i
0	0	0
0	1	0
1	0	0
1	1	1

Uma forma alternativa para o projeto de sistemas digitais complexos tal como o circuito multiplicador implica em abstrair o procedimento padrão da especificação das expressões Booleanas que o definem e implementar logicamente a semântica do sistema.

No caso do circuito multiplicador, pode-se proceder da seguinte forma:

- Observa-se que a multiplicação do multiplicando pelo multiplicador só admite duas possibilidades, ou seja, caso o bit sendo multiplicado seja "0", o resultado será uma palavra de n bits todos "0". Caso o bit sendo multiplicado seja "1", o resultado será exatamente o multiplicador;
- O bit menos significativo do resultado fica alinhado com o bit do multiplicando sendo considerado;
- no passo inicial uma soma de $n + 1$ bits deve ocorrer entre os dois primeiros resultados;
- para cada passo subsequente a soma se dá entre o resultado da multiplicação do bit sendo considerado com o resultado parcial;
- todas as somas que ocorrerão no sistema serão de $n + 1$ bits a seleção incremental dos bits do multiplicando;
- a cada passo, um bit do resultado será especificado e deve ser propagado para a saída, exceto no último passo onde todos os bits do somador devem ser deslocados para a saída;

- o bit mais significativo deve ser mapeado para o *carry out* do último somador.

A figura 9.9 apresenta o circuito do multiplicador de 4 bits implementado a partir da especificação listada anteriormente.

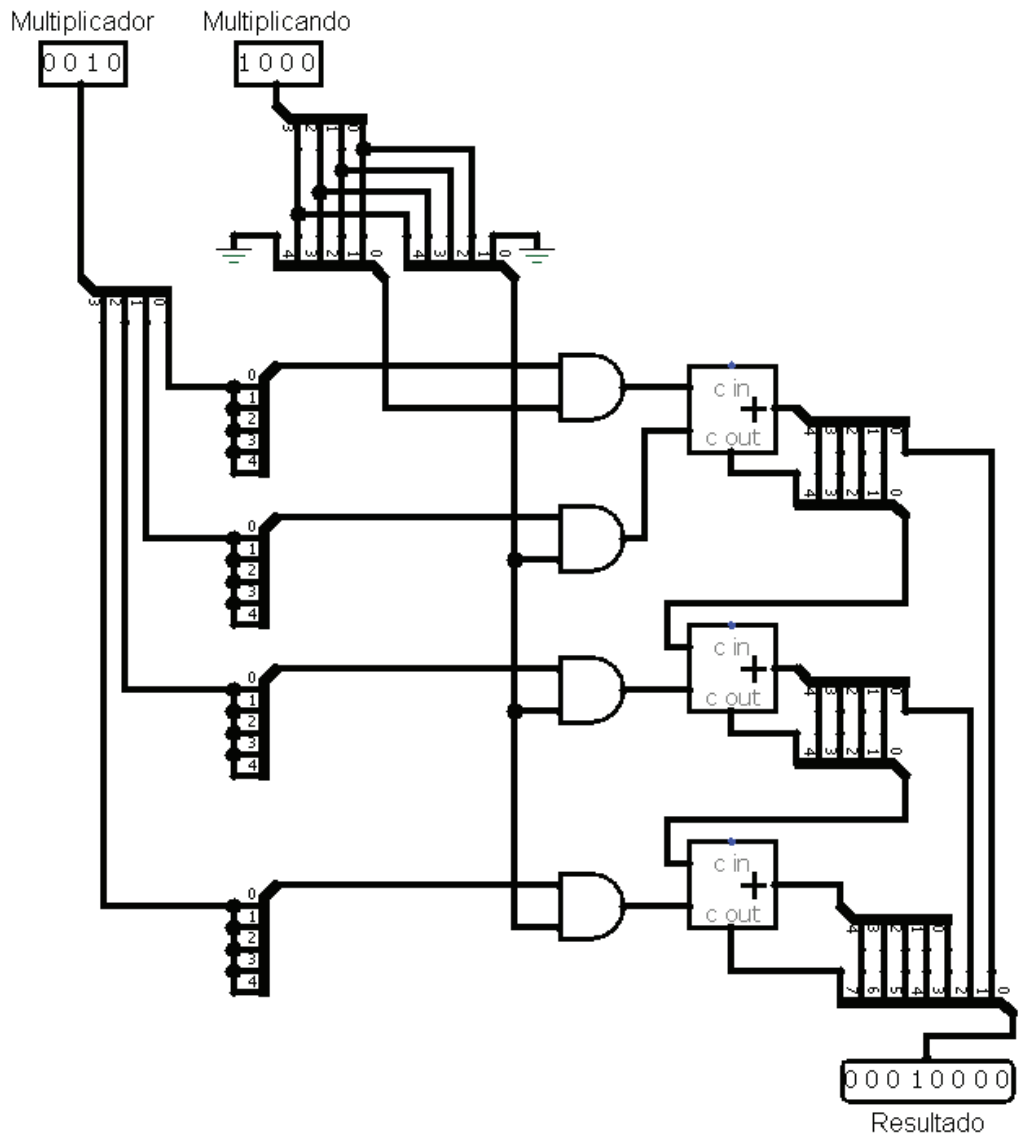


Figura 9.9: Diagrama do multiplicador de 4 bits.

Um fato interessante a se notar refere-se ao número de portas lógicas necessárias para implementar o circuito somador. Como ele utiliza diversos

somadores faz-se necessário estimar o número de portas lógicas necessárias para um somador de n bits.

Analisando a equação 9.2 observa-se que a equação da saída s_i requer duas operações OU-EXCLUSIVO. Recorde que esta operação é composta por duas inversoras, duas portas E e uma porta OU, totalizando 5 operações lógicas por OU-EXCLUSIVO e 10 para se processar s_i . A saída de *carry out* ou vai um também requer 5 portas sendo elas 3 portas E e 2 portas OU. Consequentemente, por bit, um sistema somador requer 15 portas ou um somador de n bits requererá $15 \times n$ portas totais.

O circuito multiplicador requererá $n - 1$ somadores de $n + 1$ bits cada resultado em $(n - 1) \times (15 \times n)$ portas. Adicionalmente serão necessários n arrays de portas E cada array contendo $n + 1$ portas E resultando em $n \times (n + 1)$ portas. No total um sistema multiplicador de n bits requerera $16n^2 + 14n$ portas lógicas.

Para um somador de 4 bits serão necessárias $4 \times 15 = 60$ portas. Um multiplicador de 4 bits requerera $16 \times 4^2 - 14 \times 4 = 200$ portas lógicas. O número de portas não parece tão significativo quando consideram-se palavras de 4 bits. Infelizmente o número cresce polinomialmente em função do número de bits. Para palavras de 32 bits seriam necessários 480 portas pelo somador e 15936 portas pelo multiplicador.

O que se pode dizer em relação ao atraso de propagação? Assumindo que os somadores são implementados usando a estratégia de *carry lookahead* sabemos devido as considerações anteriores que o atraso de um somador de 4 bits será de $\sigma_{SUM^n} = 1t + (n - 1) \times 3t = (1 + 9)t = 10t$. O somador de 5 bits necessário para a construção de multiplicadores de 4 bits requerem $13t$.

Para o multiplicador de 4 bits serão necessários $1t$ para todas as portas E. Adicionalmente, o primeiro somador requererá $13t$. O segundo somador só poderá computar após o primeiro terminar e assim sucessivamente. Por fim, o atraso do multiplicador será dado por $1 + (n - 1)\sigma_{SUM^{n+1}}$.

Utilizando a mesma sequencia de passos apresentadas anteriormente a figura 9.10 apresenta o diagrama de portas lógicas do circuito multiplicador de 8 bits.

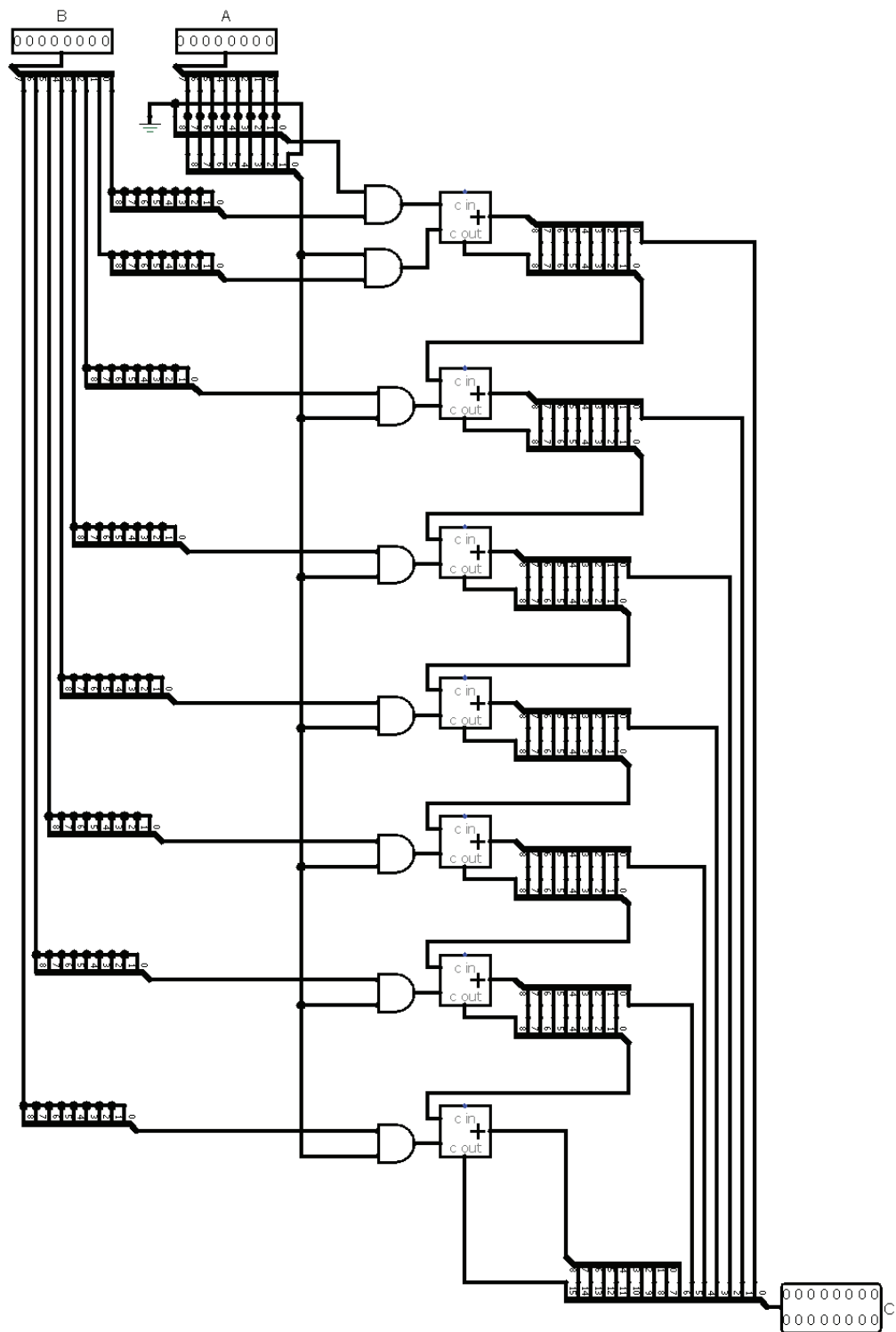


Figura 9.10: Diagrama do multiplicador de 8 bits.

Exercícios

1. Some os seguintes números em binário:
 - a) $0010\ 1010_2 + 0000\ 1111_2$
 - b) $0100\ 1100_2 + 0001\ 0101_2$
 - c) $0101\ 1100_2 + 0000\ 0101_2$
 - d) $0010\ 1010_2 + 0010\ 1010_2$
2. Subtraia os seguintes números em binário:
 - a) $0010\ 1010_2 - 1111\ 0000_2$
 - b) $0100\ 0000_2 - 0001\ 0001_2$
 - c) $0101\ 0100_2 - 1001\ 0101_2$
 - d) $1010\ 1010_2 - 0110\ 1010_2$
3. Multiplique os seguintes números em binário:
 - a) $0010\ 1010_2 \times 0000\ 0011_2$
 - b) $0100\ 1100_2 \times 0001\ 1101_2$
 - c) $0101\ 1100_2 \times 0000\ 0101_2$
 - d) $0010\ 1010_2 \times 0010\ 1010_2$
4. Divida os seguintes números em binário:
 - a) $0010\ 1010_2 \div 0000\ 0011_2$
 - b) $0100\ 1100_2 \div 0000\ 1000_2$
 - c) $0101\ 1100_2 \div 0000\ 0010_2$
 - d) $0010\ 1000_2 \div 0000\ 1010_2$
5. Some os seguintes números em complemento de 1:
 - a) $0010\ 1010_2 + 1000\ 1111_2$
 - b) $0100\ 1100_2 + 1001\ 0101_2$
 - c) $0101\ 1100_2 + 1000\ 0101_2$
 - d) $0010\ 1010_2 + 1010\ 1010_2$
6. Subtraia os seguintes números em complemento de 1:
 - a) $0010\ 1010_2 - 0000\ 1111_2$
 - b) $0100\ 1100_2 - 0001\ 0101_2$
 - c) $0101\ 1100_2 - 0000\ 0101_2$
 - d) $0010\ 1010_2 - 0010\ 1010_2$
7. O que ocorre de errado quando tentamos somar os seguintes números em complemento de 1: $01111111_2 + 00000001_2$

8. Subtraia os seguintes números usando complemento de 1:
a) $42_{10}-22_{10}$ b) $127_{10}-7_{10}$
c) $126_{10}-01_{10}$ d) $1000_{10}-999_{10}$
e) $33_{10}-31_{10}$ f) $50_{10}-55_{10}$
9. Subtraia os seguintes números usando complemento de 2:
a) $42_{10}-22_{10}$ b) $127_{10}-7_{10}$
c) $126_{10}-01_{10}$ d) $1000_{10}-999_{10}$
e) $33_{10}-31_{10}$ f) $50_{10}-55_{10}$
10. Projete usando o software Logisim ou qualquer outro para simulação de sistemas digitais os circuitos meio somador e somador completo. A seguir utilize estes subsistemas para construir somadores de 4, 8, 16 e 32 bits.
11. Utilize os somadores construídos no exercício anterior para verificar o resultado das operações do exercício 1.
12. Projete usando o software Logisim ou qualquer outro para simulação de sistemas digitais os circuitos meio somador e somador completo. A seguir utilize estes subsistemas para construir somadores de 4, 8, 16 e 32 bits.
13. Quantas portas lógicas seriam necessárias para construir um somador *ripple carry* de 64 bits? Refaça os cálculos considerando que a implementação do somador utilize a técnica de *carry lookahead*.